



Report on Co-Curricular activities done as a part of

ACXC109N

Skills in Competitive Coding

Submitted by

DEEBYANSHU JHA

24BCT0213

To

Dr. VIJAYAN E & Dr. MURALI S

Fall Semester, 2025-26

Table of Activities

S. No	<Topic – It can be events/ activities/ competition anything pertaining to the course>	Date	Hours Spent	Page No.
CO 1	To apply science, engineering, management and technology to understand real world problem		Hours: 30	Marks: 30
Activities : Participation in events like orientation session, training, guest lecture, workshop & internal competitions, organized by associated technical club/ chapter/ teams				
1	Orientation	3 rd September	2	
2	Time and space complexity	6 th January	2	
3	Strings and Arrays	7 th January	2	
4	Stacks, Queues and Linked Lists	8 th January	2	
5	Trees and Heaps	9 th January	2	
6	Graphs	10 th January	2	
7	HashMaps and HashSets	11 th January	2	
8	Dynamic Programming and Greedy Algorithms	12 th January	3	
9	Bit Manipulation, Math and Geometry	13 th January	3	
10	Prefix and Suffix Arrays	14 th January	2	
11	Prime Numbers, GDC and HCF	19 th January	2	
12	SDG: Neuracare	15 th October	2	
13	CASA: Empower, Educate, Eradicate	5 th November	2	
14	Alumni Connect	14 th December	2	
CO 2	To gain practical knowledge about best practices in chosen domain		Hours: 30	Marks: 30
Activities : Gaining of practical knowledge through participation in outreach activities, field visits and relevant events				
1	Yantra 2026	8 th -11 th February	48	
CO 3	To master team building and leadership skills		Hours: 30	Marks: 30

Activities: Participating / Organizing / Volunteering for events during graVITas organized by the associated chapter/ technical club thereby enhancing and exhibiting the team building and leadership skills Representing the institute as a part of technical Club/ Chapter/ team in events or competitions conducted by other institutes				
1	Vision to Visuals	22 nd January	2	
2	React Rumble	23 rd January	2	
3	Express Yourself	24 th January	2	
4	Dare to Dart	3 rd February	2	
5	Git Gud	4 th February	2	
6	MLtiverse Of Madness	4 th February	2	
7	Pseudo to Sudo	5 th February	2	
8	Ready Player 1	5 th February	2	
9	DEVSOC 25	8 th – 11 th February	48	
10	CookOff 10.0	28 th September	8	
11	Clueminati 3.0	25 th September	4	
Quality of the Report (format, spacing, appropriate images), uploading on time in VTOP				Marks:10

Description of Events/ Activities

Report on CodeChef-VIT Course: Session On 6th January, 2025

Date: 6th January, 2025

Instructors: Naman Bhatia & Krishita Gupta

Topics Covered:

1. Introduction to time and space complexity
2. Two Sum
3. Finding the Maximum element

1. Introduction to time and space complexity

Concept:

Time and space complexity are measures used to evaluate the efficiency of an algorithm. Time complexity refers to how the execution time of an algorithm increases with input size, while space complexity refers to the amount of memory used. These concepts help in comparing algorithms and choosing the most efficient one for a given problem.

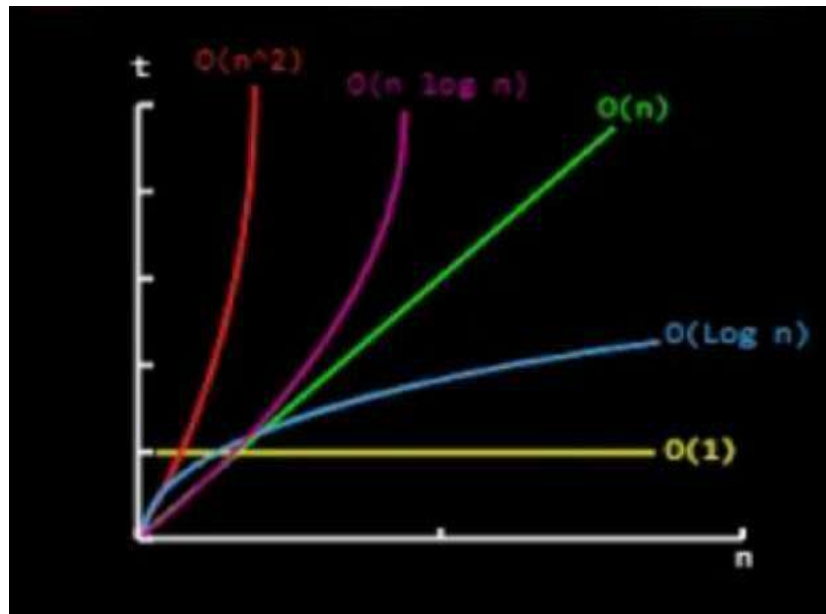
1. Understanding Time Complexity Growth Patterns

a. Explanation: Time complexity describes how the running time changes as input size increases. Common growth patterns include:

- $O(1)$ – Constant time (independent of input size)
- $O(\log n)$ – Logarithmic time (efficient for large inputs)
- $O(n)$ – Linear time (grows proportionally)
- $O(n \log n)$ – Linearithmic (common in efficient sorting)
- $O(n^2)$ – Quadratic time (slower for large inputs)

These patterns help us understand how quickly an algorithm becomes inefficient as data increases.

b. Graph:



2. Formalizing Efficiency: The Big-O Notation

- a. **Explanation:** Big-O notation is used to represent the upper bound of an algorithm's time or space complexity. It focuses on the worst-case scenario and ignores constant factors and lower-order terms. For example, an algorithm with complexity $O(n^2 + n)$ is simplified to $O(n^2)$. It provides a clear and consistent way to compare algorithm efficiency.

3. Diving into Space Complexity

a. **Explanation:**

Space complexity measures the total memory required by an algorithm, including input storage and auxiliary space. It is expressed using Big-O notation.

- **$O(1)$** – Constant space (no extra memory needed)
- **$O(n)$** – Linear space (memory grows with input size)

Efficient algorithms aim to minimize both time and space usage, but sometimes a trade-off is required.

2. Case 1: Two Sum

Concept: The Two Sum problem involves finding two numbers in an array whose sum equals a given target. While a simple approach checks all pairs with $O(n^2)$ time complexity, an efficient method uses a hash map to store elements and quickly find complements, reducing the time complexity to $O(n)$ with $O(n)$ extra space.

- **Example:**

Problem statement:

Given an array of integers and a target value, find two numbers whose sum equals the target.

The goal is to compare different approaches and analyze their efficiency

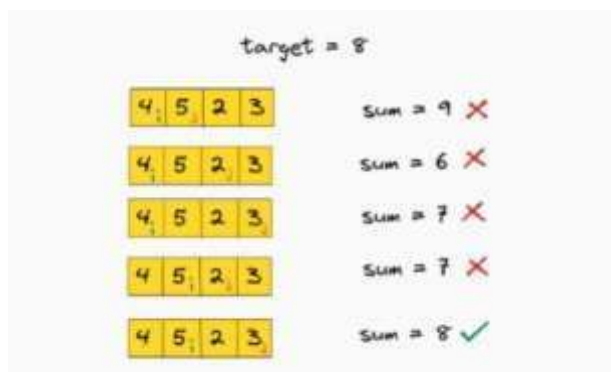
Approach 1: Brute Force

Idea:

Check every possible pair of numbers.

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$



Approach 2: Optimized Method (Hash Map)

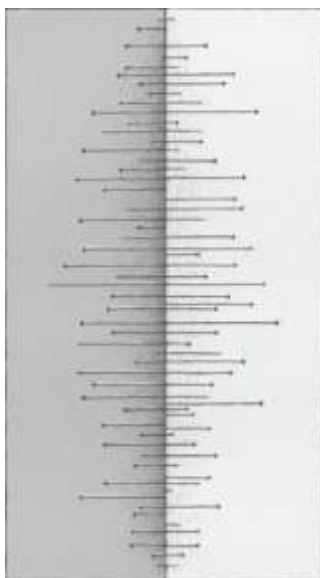
Idea:

Store previously seen values to avoid repeated work.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Demonstrates the trade-off between time and space.



3. Case Study 2: Finding the Maximum Element

- **Concept:** This problem involves identifying the largest element in a given array. The most straightforward approach is to traverse the array once, comparing each element with the current maximum. This method has a time complexity of $O(n)$ and requires constant space ($O(1)$), making it efficient and optimal for this task.

- **Example:**

Problem statement:

Given an array of numbers, find the maximum element.

Inefficient Approach:

Idea: Compare each element with every other element.

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Shows how unnecessary nested loops reduce performance.

Optimized Approach

Idea:

Traverse the array once and keep track of the maximum value.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Efficient, clean, and scalable solution.

Conclusion:

I understand that writing correct code is not enough- efficiency matters just as much. Time complexity helps me determine how fast my code runs, while space complexity shows how much memory it uses. I also realize that faster algorithms may require more memory, so I need to find the right balance between time and space to design efficient software.

Report on CodeChef-VIT Course: Session On 7th January, 2025

Date: 7th January, 2025

Instructors: Naman Goyal & Parth

Topics Covered:

1. Strings and Arrays
2. Searching
3. Sorting

1. Strings and Arrays

Concept: Arrays are linear data structures that store elements of the same type in contiguous memory locations, while strings are sequences of characters used to represent text. Both are fundamental structures used for storing and processing data efficiently.

Explanation:

Arrays allow direct access to elements using indexing, making them efficient for retrieval operations with $O(1)$ access time. However, insertion and deletion can be costly as elements may need to be shifted. Strings, often implemented as arrays of characters, support operations like concatenation, substring extraction, and comparison. Efficient handling of strings and arrays is crucial in solving problems related to searching, sorting, and data manipulation.

Example Problem – Reverse a String

Solution: The string can be reversed by swapping characters from the beginning and end moving towards the center using a two-pointer approach. This method works in $O(n)$ time and requires minimal extra space.

```
#include<iostream>
using namespace std;
class Solution {
public:
    string reverseString(string s) {
        int left = 0, right = s.length() - 1;

        while (left < right) {
            swap(s[left], s[right]);
            left++;
            right--;
        }

        return s;
    }
};
```

Example Problem – Find Maximum Element

Solution: Traverse the array while keeping track of the current maximum element. Compare each element with the current maximum and update it if a larger value is found. This approach requires a single pass through the array, resulting in $O(n)$ time complexity and $O(1)$ space complexity.

```
class Solution {
public:
    int findMax(vector<int>& arr) {
        int maxVal = arr[0];
        for (int i = 1; i < arr.size(); i++) {
            if (arr[i] > maxVal) {
                maxVal = arr[i];
            }
        }
        return maxVal;
    }
};
```

2. Searching

Concept: Searching is the process of finding a specific element in a dataset. The two most common techniques are Linear Search and Binary Search, chosen based on whether the data is sorted or not.

Explanation: Linear search checks each element one by one until the target is found, making it simple but less efficient with a time complexity of $O(n)$. Binary search, on the other hand, works only on sorted data and repeatedly divides the search space into halves, achieving a much faster time complexity of $O(\log n)$. Choosing the right searching technique improves performance significantly.

- **Linear Search:**

```
int linearSearch(vector<int>& arr, int target) {
    for (int i = 0; i < arr.size(); i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}
```

- **Binary Search:**

```
int binarySearch(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;

    while (left <= right) {
        int mid = (left + right) / 2;

        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

3. Sorting

Concept:

Sorting is the process of arranging elements in a specific order, usually ascending or descending, to organize data efficiently.

Explanation:

Basic sorting algorithms include Bubble Sort, Selection Sort, and Insertion Sort. Bubble Sort repeatedly swaps adjacent elements if they are in the wrong order. Selection Sort selects the minimum element and places it at the correct position in each iteration. Insertion Sort builds the sorted array by inserting elements into their correct position one by one. All three have a time complexity of $O(n^2)$ and are suitable for small datasets.

Example Problem – Sort an Array:

Problem: Sort a given array in ascending order.

Solution: Using Insertion Sort, elements are picked one by one and placed in their correct position within the sorted portion of the array. This method works in $O(n^2)$ time and is efficient for small or nearly sorted datasets.

```
void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Conclusion:

The concepts of searching, sorting, strings, and arrays form the foundation of problem-solving in programming. Arrays and strings provide efficient ways to store and manipulate data, while searching techniques help in quickly locating elements within that data. Sorting algorithms organize data, improving the performance of operations like searching and analysis. Together, these topics highlight the importance of choosing the right approach and optimizing time and space complexity to build efficient and scalable solutions.

Report on CodeChef-VIT Course: Session on 8th January, 2025

Date: 8th January, 2025

Instructors: Krish, Ishaan Srivastava & Sanjana Shyamsundar

Topics Covered:

1. **Stacks**
2. **Queues**
3. **Linked List**

1. **Stacks:**

- Definition and properties.
- Common operations: push, pop, top, and isEmpty.
- **Explanation:** A stack is a linear data structure that follows the Last In, First Out (LIFO) principle, meaning the last element added to the stack is the first one to be removed. Think of it like a stack of plates: you add new plates to the top, and you also remove plates from the top. Operations on a stack typically include push (to add an element), pop (to remove the top element), and peek (to view the top element without removing it). Stacks are used in various applications like managing function calls in recursion, undo operations in software, and evaluating expressions. Since elements are only added or removed from one end, stacks are efficient and easy to implement.

Applications:

- **Real-World Examples:**
 - Use cases of stacks in function call management, expression evaluation, and backtracking algorithms..

Example Problems:

1. **Balanced Parentheses:**

- **Problem:** Check if a given expression has balanced parentheses.
- **Solution:** The code checks if a string of parentheses is balanced by using a stack. It maps closing brackets to their corresponding opening brackets and iterates through the string. Opening brackets are pushed onto the stack, while for closing brackets, it checks if they match the last opening bracket in the stack. If the stack is empty or the brackets don't match, it returns false. Finally, it ensures the stack is empty at the end, indicating all brackets were properly matched and balanced.

```
def is_balanced(expression):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'
    for char in expression:
        if char in pairs.values():
            stack.append(char)
        elif char in pairs.keys():
            if stack == [] or pairs[char] != stack.pop():
                return False
    return stack == []
```

2. Queues:

- Definition and properties.
- Common operations: enqueue, dequeue and isEmpty.

- **Explanation:** A queue works like a line of people waiting for a service—new people join at the end, and the person at the front gets served first. This FIFO behavior makes queues useful in situations where order matters. Queues are widely used in scheduling tasks, handling requests, and managing resources. Since insertion and deletion happen at opposite ends, queues are efficient and simple to implement.

Applications:

- **Real-World Examples:**
 - Queues are used in CPU scheduling, printer task management, handling requests in web servers, and buffering data streams.

Example Problems:

1. Queue using Array or Linked List:

- **Problem:** Implement a queue and perform basic operations.

- **Solution:** The queue is implemented by maintaining front and rear pointers. Enqueue adds an element at the rear, while dequeue removes an element from the front. Proper checks are performed to handle underflow (empty queue) and overflow (full queue in array implementation), ensuring smooth operation of the data structure.

```
def first_non_repeating(stream):
    queue = deque()
    freq = {}

    result = ""

    for ch in stream:
        freq[ch] = freq.get(ch, 0) + 1
        queue.append(ch)

        while queue and freq[queue[0]] > 1:
            queue.popleft()

        if queue:
            result += queue[0]
        else:
            result += '#'

    return result
```

3. Linked Lists:

- Definition and properties.
- Common operations: Insertion, deletion, traversal, and searching

- **Explanation:** A linked list works by connecting nodes through pointers. Each node holds data and a link to the next node, forming a chain-like structure. Unlike arrays, linked lists do not require continuous memory, making them flexible for dynamic data. However, accessing elements is slower compared to arrays because traversal must start from the head and move sequentially. Linked lists are widely used when frequent insertions and deletions are required.

Applications:

- **Real-World Examples:**
 - Linked lists are used in memory management, implementation of stacks and queues, music playlists, browser history navigation, and dynamic data storage where size can change frequently.

Example Problems:

3. Detect Cycle in Linked List::

- **Problem:** Determine if a linked list contains a cycle..
- **Solution:** This can be solved using Floyd's Cycle Detection Algorithm (slow and fast pointers). Two pointers traverse the list, one moving one step at a time and

the other two steps. If they meet, a cycle exists; otherwise, if the fast pointer reaches null, the list has no cycle.

```
class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;

            if (slow == fast) return true;
        }
        return false;
    }
}
```

Report on CodeChef-VIT Course: Session on 9th January, 2025

Date: 9th January, 2025

Instructor: Srijani & Harika

Topics Covered: 1. Trees
2. Heaps

1. Trees

Concept: A tree is a hierarchical data structure consisting of nodes connected by edges, where one node is designated as the root and every node can have zero or more child nodes.

Explanation: Trees are used to represent hierarchical relationships such as file systems or organizational structures. Each node contains data and links to its children. Unlike linear data structures, trees allow efficient searching, insertion, and deletion operations. A common type is the binary tree, where each node has at most two children. Traversal techniques like inorder, preorder, and postorder are used to visit nodes.

Example: Search in a Binary Search Tree (BST)

Problem: Find whether a given value exists in a Binary Search Tree.

Solution: In a BST, for each node, values in the left subtree are smaller and values in the right subtree are larger. Starting from the root, compare the target

value with the current node. If it is equal, return true. If it is smaller, move to the left child; otherwise, move to the right child. This process continues until the value is found or a null node is reached. The time complexity is $O(h)$, where h is the height of the tree.

```
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = right = NULL;
    }
};

void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

2. Heaps

Concept:

A heap is a complete binary tree-based data structure that satisfies the heap property, where in a max heap the parent node is greater than its children, and in a min heap the parent node is smaller than its children.

Explanation:

Heaps are commonly used to implement priority queues. Since they are complete binary trees, they are efficiently stored using arrays. Operations like insertion and deletion maintain the heap property using heapify operations. Heaps allow efficient access to the maximum or minimum element in $O(1)$ time, while insertion and deletion take $O(\log n)$ time.

Example: Find Maximum in a Max Heap

Problem: Find the maximum element in a max heap.

Solution:

In a max heap, the largest element is always at the root node. Therefore, the maximum element can be directly accessed without traversal. This operation takes constant time $O(1)$, making heaps very efficient for priority-based operations.

```

class MaxHeap {
    vector<int> heap;

public:
    void insert(int val) {
        heap.push_back(val);
        int i = heap.size() - 1;

        // Heapify up
        while (i > 0 && heap[(i - 1) / 2] < heap[i])
            swap(heap[i], heap[(i - 1) / 2]);
        i = (i - 1) / 2;
    }

    int getMax() {
        if (heap.empty()) return -1;
        return heap[0]; // root element
    }
};

```

Report on CodeChef Course: Session on 10th January, 2025

Date: 10th January, 2025

Instructors: Edum & Rewansh

Topics Covered: 1. Graphs

1. Graphs

Concept:

A graph is a data structure consisting of a set of vertices (nodes) and edges that connect pairs of vertices.

Explanation:

Graphs are used to represent relationships between different entities, such as networks, maps, and social connections. They can be directed or undirected, weighted or unweighted. Graphs are commonly represented using adjacency lists or adjacency matrices. Traversal techniques like Breadth-First Search (BFS) and Depth-First Search (DFS) are used to explore nodes. Graphs are widely used in pathfinding, networking, and recommendation systems.

Example: Graph Traversal using BFS

Problem: Traverse all nodes of a graph starting from a given node.

Solution:

In Breadth-First Search (BFS), we explore nodes level by level using a queue. Starting from the source node, we mark it as visited and push it into the queue. Then, we repeatedly remove a node from the queue, visit it, and add all its unvisited neighbors to the queue. This process continues until all reachable nodes are visited. The time complexity is $O(V+E)$, where V is vertices and E is edges.

```

int main() {
    int n = 5;
    vector<vector<int>> adj(n);
    adj[0] = {1, 2};
    adj[1] = {0, 3};
    adj[2] = {0, 4};
    adj[3] = {1};
    adj[4] = {2};
    vector<bool> visited(n, false);
    queue<int> q;
    int start = 0;
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int node = q.front();
        q.pop();
        cout << node << " ";
        for (int neighbor : adj[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
    return 0;
}

```

Conclusion:

Trees, heaps, and graphs are essential non-linear data structures used to represent complex relationships and hierarchical data. Trees provide an efficient way to organize data hierarchically, heaps are useful for priority-based operations, and graphs model real-world networks and connections. Understanding these structures and their operations helps in solving advanced problems efficiently, with optimized time and space complexity.

Report on CodeChef-VIT Course: Session on 11th January, 2025

Date: 11th January, 2025

Instructors: Thomas & Shaayoon

Topics Covered: 1. HashSets
2. HashMaps

1. HashSets

Concept:

A HashSet is a data structure used to store unique elements, where duplicates are not allowed. It is based on hashing, which allows fast insertion, deletion, and lookup operations.

Explanation:

HashSets use a hash function to map elements to specific positions, enabling average

time complexity of $O(1)$ for operations like add, remove, and search. Since elements are stored based on hash values, the order is not maintained. HashSets are useful when checking for duplicates or ensuring uniqueness in data.

Example: Check for Duplicates in an Array

Problem: Determine if an array contains duplicate elements.

Solution:

Traverse the array and insert each element into a HashSet. If an element is already present in the set, it means a duplicate exists. This approach efficiently detects duplicates in $O(n)$ time using $O(n)$ extra space.

```
bool hasDuplicate(vector<int>& arr) {  
    unordered_set<int> s;  
  
    for (int num : arr) {  
        if (s.count(num)) return true;  
        s.insert(num);  
    }  
    return false;  
}
```

2. HashMaps

Concept: A HashMap is a data structure that stores data in key-value pairs, where each key is unique and maps to a specific value. It uses hashing to enable fast access to data.

Explanation: HashMaps use a hash function to compute an index for storing values, allowing average time complexity of $O(1)$ for insertion, deletion, and lookup. They are widely used for counting frequencies, mapping relationships, and efficient data retrieval. However, elements are not stored in any sorted order.

Example: Frequency Count of Elements

Problem: Count the frequency of each element in an array.

Solution: Traverse the array and store each element as a key in the HashMap. If the element already exists, increment its count; otherwise, initialize it with 1. This approach efficiently counts frequencies in $O(n)$ time.

```
void frequencyCount(vector<int>& arr) {  
    unordered_map<int, int> mp;  
  
    for (int num : arr) {  
        mp[num]++;  
    }  
  
    for (auto it : mp) {  
        cout << it.first << " -> " << it.second << endl;  
    }  
}
```

Conclusion:

HashSet and HashMap are efficient data structures based on hashing that enable fast data storage and retrieval. While HashSet is used to store unique elements and avoid duplicates, HashMap stores key-value pairs for mapping and frequency-based problems. Both provide average time complexity of $O(1)$ for basic operations, making them highly useful for optimizing algorithms and handling large datasets efficiently.

Report on CodeChef-VIT Course: Session on 12th January, 2025

Date: 12th January, 2025

Instructors: Yogesh Kumar & Aarav Jha

Topics Covered: 1. Dynamic Programming
2. Greedy Algorithms

1. Dynamic Programming

Concept:

Dynamic Programming (DP) is an optimization technique used to solve problems by breaking them into smaller overlapping subproblems and storing their results to avoid repeated computations.

Explanation:

DP is used when a problem exhibits overlapping subproblems and optimal substructure. Instead of solving the same subproblem multiple times, results are stored (memoization) or built iteratively (tabulation). This significantly reduces time complexity compared to recursive approaches. DP is widely used in problems like Fibonacci, knapsack, and shortest path algorithms.

Example: Fibonacci Number

Problem: Find the n th Fibonacci number.

Solution:

Instead of using a recursive approach with exponential time, DP stores previously computed values. Starting from base cases, each Fibonacci number is calculated using the sum of the previous two values. This reduces time complexity to $O(n)$ with $O(n)$ space.

```
int fibonacci(int n) {
    vector<int> dp(n + 1);
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}
```

2. Greedy Algorithms

Concept: Greedy algorithms solve problems by making the best choice at each step with the hope of finding the global optimum.

Explanation: In greedy algorithms, decisions are made locally without considering future consequences. These algorithms are efficient and simple, often used when a problem has the greedy choice property and optimal substructure. Examples include activity selection, coin change (for certain denominations), and minimum spanning tree algorithms.

Example: Activity Selection Problem

Problem: Select the maximum number of non-overlapping activities from a set of activities with start and end times.

Solution: Sort the activities based on their finishing times. Always select the activity that finishes earliest, then move to the next activity whose start time is after the last selected activity's end time. This approach ensures the maximum number of activities is selected with a time complexity of $O(n \log n)$ due to sorting.

```
int activitySelection(vector<pair<int,int>>& arr) {
    sort(arr.begin(), arr.end(), [](auto &a, auto &b) {
        return a.second < b.second;
    });

    int count = 1;
    int lastEnd = arr[0].second;

    for (int i = 1; i < arr.size(); i++) {
        if (arr[i].first >= lastEnd) {
            count++;
            lastEnd = arr[i].second;
        }
    }

    return count;
}
```

Conclusion:

Greedy algorithms and Dynamic Programming are both important techniques for solving optimization problems efficiently. Greedy algorithms make locally optimal choices at each step and are simple and fast, but they do not always guarantee the best global solution. In contrast, Dynamic Programming considers all possible subproblems and stores results to ensure an optimal solution, though it may use more time and space. Choosing between them depends on the problem's properties, such as whether it satisfies the greedy choice property or requires overlapping subproblems handling.

Report on CodeChef-VIT Course: Session on 13th January, 2025

Date: 13th January, 2025

Instructors: Ayman & Aditya

Topics Covered:

1. Bit Manipulation
2. Maths
3. Geometry

1. Bit Manipulation

Concept: Bit manipulation involves performing operations directly on binary representations of numbers using bitwise operators like AND, OR, XOR, and NOT.

Explanation: Bit manipulation is used to optimize algorithms by reducing time and space complexity. Operations like checking if a number is even/odd, setting, clearing, or toggling bits can be done efficiently using bitwise operators. These operations are faster than arithmetic operations and are widely used in low-level programming, competitive programming, and optimization problems.

Example: Check if a Number is Power of Two

Problem: Determine if a given number is a power of 2.

Solution: A number is a power of 2 if it has only one set bit in its binary representation. Using the expression $n \& (n-1) == 0$, if the result is 0, then the number is a power of 2. This works because subtracting 1 flips all bits after the rightmost set bit. The time complexity is $O(1)$.

Example Problems:

• Count Set Bits:

- Problem: Count the number of 1s in the binary representation of a number.
- Solution:

```
def count_set_bits(n):  
    count = 0  
    while n:  
        count += n & 1  
        n >>= 1  
    return count
```

2. Maths

Concept: Mathematics in competitive coding involves using mathematical concepts and formulas to solve problems efficiently and optimize computations.

Explanation: Common topics include number theory, modular arithmetic, prime numbers, GCD, LCM, and combinatorics. These concepts help in reducing time complexity and handling large inputs. Techniques like fast exponentiation, modulo operations, and divisibility rules are widely used to avoid overflow and improve performance.

Example: Compute GCD of Two Numbers

Problem: Find the greatest common divisor (GCD) of two integers.

Solution: The GCD can be efficiently calculated using the Euclidean Algorithm, where we repeatedly replace the larger number with the remainder of the division until one number becomes zero. The remaining non-zero number is the GCD. This method works in $O(\log_{10} n)$ time.

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int temp = b;  
        b = a % b;  
        a = temp;  
    }  
    return a;  
}
```

3. Geometry

Concept: Geometry in competitive coding involves using mathematical properties of shapes, points, and lines to solve problems related to coordinates, distances, and areas.

Explanation: Common topics include distance between two points, slope of a line, area of shapes, and orientation of points. These concepts are often used in problems

involving coordinates and graphs. Efficient formulas and geometric reasoning help in solving problems accurately with low time complexity.

Example: Distance Between Two Points

Problem: Find the distance between two points in a 2D plane.

Solution: The distance is calculated using the distance formula derived from the Pythagorean theorem:

$$\text{distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

This can be computed in constant time $O(1)$

Conclusion:

Geometry, mathematics, and bit manipulation are essential tools in competitive coding that help in solving problems efficiently and accurately. Mathematical concepts provide the foundation for optimizing calculations and handling large inputs, while geometry helps in solving coordinate-based and spatial problems. Bit manipulation offers fast and memory-efficient solutions by working directly at the binary level. Together, these topics enhance problem-solving skills and enable the development of highly optimized algorithms.

Report on CodeChef-VIT Course: Session on 14th January, 2025

Date: 14th January, 2025

Instructor: Rehan & Somasekar

Topics Covered: 1. Prefix and Suffix Arrays

1. Prefix and Suffix Arrays

Concept: Prefix and suffix arrays are techniques used to store cumulative information about an array, helping to answer range-based queries efficiently.

Explanation: A prefix array stores the sum (or result) of elements from the beginning up to a certain index, while a suffix array stores the sum from a certain index to the end. These arrays allow quick computation of subarray sums and reduce time complexity from $O(n)O(n)O(n)$ per query to $O(1)O(1)O(1)$ after preprocessing. They are widely used in range queries, cumulative sums, and optimization problems.

Example: Range Sum Query

Problem: Find the sum of elements between two indices in an array.

Solution: First, construct a prefix sum array where each index stores the sum up to that point. Then, the sum between indices l and r can be calculated as:

$$\text{sum} = \text{prefix}[r] - \text{prefix}[l - 1]$$

This allows answering queries in constant time $O(1)$ after $O(n)$ preprocessing.

```
vector<int> prefixSum(vector<int>& arr) {  
    int n = arr.size();  
    vector<int> prefix(n);  
  
    prefix[0] = arr[0];  
    for (int i = 1; i < n; i++) {  
        prefix[i] = prefix[i - 1] + arr[i];  
    }  
  
    return prefix;  
}
```

Conclusion:

Prefix and suffix arrays are powerful techniques for optimizing range-based computations in arrays. By preprocessing cumulative information, they allow queries like subarray sums to be answered in constant time, significantly improving efficiency. These methods are widely used in competitive programming to reduce time complexity and handle large datasets effectively.

Report on CodeChef Course: Session on 19th January, 2025

Date: 19th January, 2025

Instructors: Edum & Rehan

Topics Covered: 1. Prime Numbers
2. GDC/HCF

1. Prime Numbers

Concept:

A prime number is a natural number greater than 1 that has exactly two factors: 1 and itself.

Explanation:

Prime numbers are fundamental in mathematics and widely used in algorithms, especially in number theory and cryptography. To check if a number is prime, we test divisibility from 2 up to n . If no number divides it, the number is prime. This reduces time complexity to $O(n^{1/2})$.

Example: Check if a Number is Prime

Problem: Determine whether a given number is prime.

Solution:

Check if the number is less than or equal to 1 (not prime). Then iterate from 2 to $n^{1/2}$; if any number divides it, it is not prime. Otherwise, it is prime.

```
bool isPrime(int n) {
    if (n <= 1) return false;

    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```

2. GDC/HCF

Concept: GCD (Greatest Common Divisor) or HCF (Highest Common Factor) is the largest number that divides two or more numbers without leaving a remainder.

Explanation: GCD/HCF is commonly used in problems involving divisibility, fractions, and number simplification. The most efficient way to compute it is using the Euclidean Algorithm, where we repeatedly replace the larger number with the remainder of the division until one number becomes zero. The remaining number is the GCD. This method works in $O(\log n)$ time.

Example: Find GCD of Two Numbers

Problem: Find the GCD of two integers.

Solution: Apply the Euclidean Algorithm by repeatedly performing $a \% b$ until $b=0$. The final value of a is the GCD.

```
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

Conclusion:

Prime numbers and GCD/HCF are fundamental concepts in mathematics that play a crucial role in problem-solving and algorithm design. Prime numbers help in understanding number properties and are widely used in areas like cryptography, while GCD/HCF is essential for simplifying problems involving divisibility and ratios. Together, they form the basis of many efficient algorithms in competitive coding and mathematical computations.

Report on CodeChef Course: Session on 15th Oct

Date: 15th October 2025

Session Name: SDG: Neuracare

Event Description:

Join us for an insightful session on *Neuracare*, an innovative solution designed to transform education through personalized learning. This project addresses the challenges faced by neurodivergent students by leveraging AI-driven technologies to create adaptive study plans, interactive learning modules, and emotional support systems. The session explores how traditional “one-size-fits-all” education can be reimaged to be more inclusive, accessible, and effective. By integrating features like progress tracking, focus tools, and peer support systems, Neuracare aims to bridge the gap between education and mental well-being, aligning with Sustainable Development Goals focused on quality education and inclusivity.

Event Summary:

The *Neuracare* session introduced an AI-powered personalized learning platform designed for neurodivergent students, addressing the limitations of traditional one-size-fits-all education. Participants explored key features such as adaptive study plans, interactive learning modules, focus tools, and emotional support systems that promote both academic growth and well-being. The session also highlighted the platform’s social impact in improving accessibility, fostering inclusivity, and empowering students, while demonstrating how technology can bridge gaps in education and mental health.

Report on CodeChef Course: Session on 5th November, 2024

Date: 5th November, 2025

Session Name: CASA: Empower, Educate, Eradicate

Event Description:

CodeChef-VIT is on a mission to transform how our nation addresses addiction by raising awareness amongst young individuals. We'll dig deep into the physical and mental wreckage caused by drug abuse and equip you with vital information on available treatment services. Whether you're passionate about prevention or lending support to your loved ones, let's stand together to build a healthier, more secure world. We extend our hand to you, inviting you to unite, inviting you to CASA.

Event Summary:

CodeChef-VIT’s CASA initiative focused on raising awareness about addiction and its impact on physical and mental well-being, while equipping young individuals with essential knowledge on prevention and access to treatment resources. The session, conducted by Ms. Jessica Jesudassan, provided valuable insights into understanding addiction, fostering healthier habits, and offering support to those affected, complemented by an engaging ice-breaking activity that encouraged interaction and open dialogue among participants. Overall, the event brought attendees together to promote awareness, empathy, and a shared commitment to building a healthier and more supportive community.

Report on Alumni Connect - CodeChef Mentoring Session: Event on December 14th, 2025

Event Description:

Alumni Connect was a national-level mentoring session organized under the Innovation and Technical categories, aimed at providing students with valuable industry insights. The event featured Amulya Boyapati, a Software Engineer at Apple Inc., who shared her experiences and expertise in the field of software engineering.

The session offered an in-depth perspective on real-world applications, emerging trends in data systems, and the evolving landscape of AI-driven engineering. Through this interactive seminar, participants gained practical knowledge, career guidance, and a deeper understanding of the industry, making it an engaging and insightful learning experience.

Event Summary:

Alumni Connect was a national-level mentoring session featuring Amulya Boyapati, Software Engineer at Apple Inc. The session provided insights into software engineering, data systems, and AI-driven technologies, offering participants valuable industry exposure and guidance.

Report on CodeChef Course: Session on 21st January, 2026

Date: 21st January, 2026

Session 1: The Pre-Hack Ritual

Event Description:

Step into the dynamic world of hackathons, where creativity and collaboration come together to drive innovation. This session introduces you to how hackathons function and how they can accelerate your learning and growth. Discover how to generate meaningful ideas, deliver compelling pitches, form effective teams, and sharpen your strategy with guidance from experts. Gear up to approach your next hackathon with confidence and a winning mindset.

Event Summary:

The seminar provided an overview of hackathons, including brainstorming concepts, creating effective pitches, developing strong teams, and refining methods. The program included interactive activities and a Q&A session to answer questions. Participants received useful insights and were prepared to approach future hackathons with confidence.

Report on CodeChef Course: Session on 22nd January, 2026

Date: 22nd January, 2026

Session 1: Vision to Visuals

Event Description:

Explore the art of design in our hands-on UI-UX foundations workshop, where you'll create intuitive, engaging, and visually attractive interfaces. Discover the key principles of user experience and interface design, with a focus on seamless digital interactions. Whether you're a beginner discovering design or an innovator boosting creativity, join us to bring ideas to life with precision and style!

Event Summary:

The UI-UX program included a hands-on Figma workshop, which taught attendees how to create intuitive and visually appealing interfaces. Attendees looked at user experience principles and interface design, with a focus on smooth digital interactions. They developed practical skills in Figma, learning how to turn ideas into polished creations with creativity and precision.

Report on CodeChef Course: Session on 23rd January, 2026

Date: 23rd January, 2026

Session 1: React Rumble

Event Description:

Join us for a hands-on web development class with Next.js. Learn the fundamentals of setting up and creating code, and then discover how to make your website dynamic with props and events. Use hooks such as `useState` and `useEffect` to create gorgeous pages with Tailwind CSS. Finally, connect everything via routing. This class is ideal for anyone looking to take their initial steps in web programming!

Event Summary:

This hands-on seminar taught participants how to develop dynamic webpages with Next.js. From project setup to developing interactive code with props and events, attendees learned about the power of hooks like `useState` and `useEffect`, as well as how to create attractive designs using Tailwind CSS. The talk also discussed seamless routing, which connects everything together to create completely functional online apps. The workshop helped participants take their initial steps into web development, preparing them to create meaningful and interactive digital experiences.

Report on CodeChef Course: Session on 24th January, 2026

Date: 24th January, 2026

Session 1: Express Yourself

Event Description:

Join us for an interactive session on Express.js, one of the most popular frameworks for building backend applications with Node.js. In this hands-on workshop, you'll learn how to set up a server, create RESTful APIs, and connect your application to a database. By the end, you'll have built a functional backend system while understanding key concepts like routing, middleware, and request handling. Ideal for beginners and anyone looking to strengthen their backend development skills!

Event Summary:

This hands-on session guided participants through the fundamentals of backend development using Express.js, a powerful Node.js framework. Attendees learned how to set up a server, structure a project, and build RESTful APIs from scratch while understanding core concepts like routing, middleware, and request-response handling. The workshop also introduced database integration, enabling participants to connect their application to a database and perform basic CRUD operations.

Report on CodeChef Course: Session on 3rd February, 2026

Date: 3rd February, 2026

Session 1: Dare to Dart

Event Description:

Unleash the full potential of Flutter, the breakthrough framework for cross-platform app development! In this hands-on session, you'll learn how to create beautiful, high-performance apps, design seamless user interfaces, and retrieve dynamic data via API requests. Perfect for both beginners and developers, improve your abilities and imagination to become a Flutter expert!

Event Summary:

Flutter, a cross-platform app development framework, will be the focus of this interactive class. Hands-on training in which they showed how to develop high-performance apps, build intuitive user interfaces, and implement API calls for dynamic data integration. The session gave participants hands-on experience while also inspiring creativity. By the end of the course, attendees were well-prepared to use Flutter's features and improve their careers as app developers.

Report on CodeChef Course: Session on 4th February, 2026

Date: 4th February, 2026

Session 1: Git Gud

Event Description:

Git, the fundamental tool for monitoring changes and smooth collaboration, is the perfect place to start with version control. This webinar covers everything from the foundations to complex procedures, preparing you to commit, branch, and merge with confidence. Whether you're a newbie or an advanced user, it's time to Git Set Go!

Event Summary:

The Git lesson covered the fundamentals of version control, such as committing, branching, and merging. Participants downloaded and installed Git, then performed important commands through hands-on exercises. The event laid a solid basis for successful collaboration and development.

Session 2: MLtiverse Of Madness

Event Description:

Join us for an interactive workshop exploring the amazing world of Machine Learning! Learn the principles of supervised, unsupervised, and reinforcement learning, as well as the mathematics underlying these algorithms and how to write them from scratch. This session will get you ready to use ML in your projects and prepare you for the forthcoming hackathon. Perfect for novices and tech lovers, join us as we turn concepts into meaningful applications and prepare for the future of innovation!

Event Summary:

The hands-on Machine Learning session taught participants the foundations of supervised, unsupervised, and reinforcement learning. The workshop went over fundamental mathematical principles and included real coding activities. It taught significant abilities for effectively applying Machine Learning to real-world tasks.

Report on CodeChef Course: Session on 5th February, 2026

Date: 5th February, 2026

Session 2: From Pseudo to Sudo

Event Description:

Discover the secrets of cybersecurity with our workshop. Discover encryption, safe authentication, and new technologies such as IoT, AI, and blockchain. Hands-on activities and real-world use cases will teach you how to design robust systems, protect data, and create inventive hackathon projects. Whether you're a newbie or an experienced coder, this class will provide you with the tools you need to succeed at hackathons and beyond.

Event Summary:

"CyberSiege" workshop in the Ambedkar Auditorium. Through hands-on activities and real-world use cases, attendees learned about encryption, secure authentication, and cutting-edge technologies like IoT, AI, and blockchain. Attendees learned how to build robust systems, secure data, and create new hackathon projects. The session provided attendees with the necessary skills and information to succeed in cybersecurity, hackathons, and other endeavors.

Report on CodeChef Course: Session on 6th February, 2026

Date: 6th February, 2026

Session 2: Ready Player 1

Event Description:

Gather, Gifted Groundbreakers: it is time to gain ground, gear up, and boldly break into the Great GAMEDEV Domain. In this class, we will explore in depth at game development, its applications, and gain hands-on experience producing a game using the Unreal Engine. Whether you are inquisitive or just starting out on your development adventure, these workshops will teach you the fundamentals of building assets and generating code using a combination of C and blueprints.

Event Summary:

This program covered the principles of game development, its various applications, and provided hands-on experience in creating a game using Unreal Engine. Attendees obtained practical insights and skills in a variety of areas, including asset assembly and game logic development using C and blueprints. Participants departed feeling prepared and inspired to explore the possibilities of game development.

Report on CodeChef Course: Event from 8-11 February, 2026

Date: 8th February, 2026 - 11th February, 2026

Event: DevSoc' 26

Event Description:

DevSOC (Developers' Sprint Of Code), in its seventh edition, is the flagship annual event by CodeChef-VIT, aiming at empowering young minds by fostering meaningful connections between tech enthusiasts and innovators. This 48-hour sprint-style hackathon is designed to encourage creativity, collaboration, and problem-solving in an intense yet engaging environment. Spanning over four days, DevSOC'26 provides the ultimate platform for participants to Build. Beyond. Limits.

Event Summary:

DevSOC'26 brought together 1,000+ participants for an intense and collaborative hackathon experience, fostering innovation and real-world problem-solving. With five diverse tracks—Digital Economy, Open Innovation, Tech for Good, Media & Entertainment, and the Hotfoot AI Challenge—participants explored a wide range of domains to build impactful solutions.

Supported by sponsors Hotfoot and Backboard.io, the event featured a prize pool exceeding ₹2.5 lakhs, motivating teams to push boundaries and deliver high-quality projects. DevSOC'26 successfully created a competitive yet engaging environment where creativity, collaboration, and technical excellence thrived.

Report on CodeChef graVITas'25 Events:

1) CookOff 10.0: Event on 28th September, 2025

Date: 28th September, 2025

Event Description:

CookOff 10.0 marked the tenth edition of CodeChef VIT's flagship competitive programming contest, celebrating a decade of innovation, problem-solving, and technical excellence. Organized by CodeChef-VIT, the event continued its legacy of fostering a strong coding culture and bringing together passionate programmers.

The contest was held on September 28th at Anna Auditorium, running from 8:00 AM to 3:00 PM. Designed to deliver an intense and engaging experience, CookOff 10.0 challenged participants through three competitive rounds that tested their analytical thinking, speed, and problem-solving skills.

The competition was conducted on a dedicated coding portal developed by CodeChef, ensuring a seamless, interactive, and technologically advanced experience for all

participants. CookOff 10.0 stood as a testament to the growth and impact of competitive programming within the student community.

Event Summary:

CookOff 10.0, CodeChef VIT's flagship competitive programming contest, was successfully held on September 28th at Anna Auditorium. The event featured three rigorous rounds conducted on a dedicated CodeChef platform, ensuring a seamless experience. It witnessed strong participation and upheld its legacy of promoting excellence in competitive programming.

2) Clueminati 3.0: Event on 25th September, 2025

Date: 25th September, 2025

Event Description:

Clueminati 3.0, the third edition of CodeChef-VIT's immersive treasure hunt event, delivered a thrilling experience filled with mystery, strategy, and storytelling. Held on September 25th at Anna Auditorium, the event combined interactive gameplay with a compelling narrative, challenging participants to navigate a world divided between Earth and Hell.

Centered around a storyline of conflict, sacrifice, and survival, teams were split into groups that tackled parallel challenges across indoor and outdoor settings. Participants engaged in solving riddles, uncovering clues, and completing strategic tasks, all while progressing through a two-round format designed to test their wit, coordination, and problem-solving abilities.

With its unique blend of narrative depth and dynamic gameplay, Clueminati 3.0 provided an engaging and immersive experience, pushing participants to collaborate effectively and think critically under pressure.

Event Description:

Clueminati 3.0, organized by CodeChef-VIT, was successfully conducted on September 25th at Anna Auditorium. The event featured a two-round treasure hunt format with teams divided into indoor and outdoor groups, engaging in parallel challenges. Combining immersive storytelling with strategic gameplay, the event witnessed enthusiastic participation and delivered a dynamic and engaging experience.